Guia de consulta — XMAC02

Compilado dos slides, notebooks e atividades desta pasta. Reúne fórmulas, critérios de escolha, padrões de Python e um mapa dos arquivos originais. Os CSVs são bases de exercícios; seus registros não foram reproduzidos aqui.

Escopo da disciplina

O slide introdutório apresenta a disciplina **Métodos Matemáticos para Análise de Dados (XMAC02)**, com fundamentos estatísticos e probabilísticos, ferramentas computacionais em Python e tópicos de álgebra linear. A ementa cita estatística descritiva, probabilidade, análise combinatória, variáveis aleatórias, distribuições, amostragem, testes de hipótese e álgebra linear (sistemas lineares, eliminação gaussiana e regressão polinomial). Os materiais localizados desenvolvem principalmente Python, estatística e probabilidade; não foram encontrados notebooks específicos de álgebra linear.

O slide também registra avaliações e datas de um período letivo específico. Como o calendário pode estar desatualizado, consulte o professor para informações atuais.

Consulta rápida

Se a questão pede...	Consulte
Média, mediana, moda, dispersão ou boxplot	Estatística descritiva
Chance de eventos, contagem de combinações	Probabilidade e combinatória
Número de sucessos em n tentativas	Binomial
Número de ocorrências em um intervalo	Poisson
Peso, altura, tempo ou medida contínua	Normal e exponencial
Média baseada em amostras	Amostragem e TCL
Uma média ou comparação de duas médias	Escolha do teste
Proporção, variância ou três ou mais médias	Testes específicos
Ler CSV, filtrar, agrupar ou plotar	Python para análise de dados

1. Estatística descritiva

Tendência central

- **Média:** soma dos valores dividida pela quantidade: $\bar{x} = \Sigma x_i / n$. Sensível a valores extremos.
- **Mediana:** valor central após ordenar. Com quantidade par, é a média dos dois valores centrais. Mais resistente a extremos.
- **Moda:** valor ou categoria mais frequente. Pode haver mais de uma moda ou nenhuma moda única.

Dispersão e posição

- **Amplitude:** $máximo - mínimo$.
- **Variância populacional:** $\sigma^2 = \Sigma(x_i - \mu)^2 / N$.
- **Variância amostral:** $s^2 = \Sigma(x_i - \bar{x})^2 / (n - 1)$.
- **Desvio padrão:** raiz quadrada da variância; expresso na mesma unidade dos dados.
- **Quartis:** Q1, Q2 (mediana) e Q3 dividem os dados ordenados em quatro partes.
- **Amplitude interquartil:** $IQR = Q3 - Q1$; resume a dispersão dos 50% centrais. Em boxplots, pontos muito afastados podem indicar valores atípicos.
- **Quantis:** cortes da distribuição; por exemplo, decis dividem em 10 partes e percentis em 100.

No pandas, `Series.std()` e `Series.var()` usam por padrão a fórmula **amostral** (`ddof=1`). `Series.count()` não conta valores ausentes.

```
s = df["coluna"]
s.mean()
s.median()
s.mode()
s.std()
s.var()
s.min(), s.max()
s.quantile([0.25, 0.50, 0.75])
```

Uma distribuição aproximadamente normal costuma ter cerca de 68% dos valores entre $\mu \pm 1\sigma$, 95% entre $\mu \pm 2\sigma$ e 99,7% entre $\mu \pm 3\sigma$.

2. Probabilidade e combinatória

Conceitos

- **Experimento aleatório:** procedimento cujo resultado não se conhece de antemão.
- **Espaço amostral:** conjunto de todos os resultados possíveis.
- **Evento:** subconjunto do espaço amostral.
- **União $A \cup B$:** A ou B ocorre (ou ambos).
- **Intersecção $A \cap B$:** A e B ocorrem simultaneamente.
- **Complemento:** $P(A^c) = 1 - P(A)$.
- **Mutuamente exclusivos:** não podem ocorrer juntos; $P(A \cap B)=0$.
- **Independentes:** a ocorrência de um não altera a chance do outro; $P(B|A)=P(B)$.

Regras principais

- Probabilidade clássica, quando os resultados são equiprováveis:
  $P(A) = \text{casos favoráveis} / \text{casos possíveis}$.
- Regra da adição: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.
- Regra da multiplicação: $P(A \cap B) = P(A) \times P(B|A)$.
- Se A e B são independentes: $P(A \cap B) = P(A) \times P(B)$.
- Em retiradas **sem reposição**, as probabilidades seguintes mudam: os eventos são dependentes.

Permutações e combinações

Se n itens são considerados e r são escolhidos:

- **Ordem importa, sem repetição:** $nPr = n! / (n-r)!$.
- **Ordem não importa, sem repetição:** $nCr = n! / [r!(n-r)!]$.
- **Ordem importa, com repetição:** n^r.
- **Escolha sem ordem, com repetição:** $C(n+r-1, r)$.

Use permutação para cargos distintos (presidente, vice, secretário); combinação para selecionar um grupo sem cargos.

3. Distribuições de probabilidade

Uma distribuição associa probabilidades aos valores possíveis de uma variável aleatória. A soma das probabilidades é 1.

Binomial — contagem de sucessos

Use quando há n tentativas, dois resultados por tentativa, probabilidade de sucesso p constante e tentativas independentes.

- $P(X=x) = C(n,x)\ p^x\ (1-p)^{n-x}$
- Média: $\mu = np$
- Variância: $\sigma^2 = np(1-p)$
- Desvio padrão: $\sigma = \sqrt{np(1-p)}$

Em SciPy: `pmf` é probabilidade exata; `cdf(k)` é $P(X \le k)$; `sf(k)` é $P(X > k)$.

```
from scipy.stats import binom

binom.pmf(2, n=20, p=0.12)  # exatamente 2 sucessos
binom.cdf(2, n=20, p=0.12)  # no máximo 2
binom.sf(1, n=20, p=0.12)   # 2 ou mais
binom.mean(20, 0.12)
binom.std(20, 0.12)
```

Poisson — ocorrências em um intervalo

Use para contar eventos aleatórios em uma unidade fixa (tempo, área, distância), quando a taxa média λ é conhecida e as ocorrências são independentes.

- $P(X=x) = e^{-\lambda}\ \lambda^x\ /\ x!$
- Média e variância: λ.

λ deve corresponder ao intervalo da pergunta. Se a taxa é 3,6 clientes por 10 minutos, para esse intervalo use $\lambda=3.6$; para outro intervalo, ajuste proporcionalmente.

```python
from scipy.stats import poisson

poisson.pmf(7, mu=3.6)  # exatamente 7 ocorrências
poisson.cdf(7, mu=3.6)  # até 7
poisson.sf(6, mu=3.6)   # 7 ou mais
```

Normal — variável contínua simétrica

Definida pela média μ e desvio padrão σ; é simétrica e tem área total 1. Padronização: $z = (x-\mu)/\sigma$.

- $P(a < X < b) = F(b) - F(a)$.
- $P(X > a) = 1 - F(a)$, também calculável com a função de sobrevivência.
- Em variável contínua, $P(X = a)=0$; portanto $P(X\leq a)=P(X<a)$.
- Para obter o valor correspondente a um percentil, use a inversa da CDF (`ppf`).

```python
from scipy.stats import norm

norm.cdf(x, loc=mu, scale=sigma)       # P(X <= x)
norm.cdf(b, loc=mu, scale=sigma) - norm.cdf(a, loc=mu, scale=sigma)
norm.sf(x, loc=mu, scale=sigma)        # P(X > x)
norm.ppf(0.95, loc=mu, scale=sigma)    # percentil 95
```

Exponencial — tempo até o próximo evento

Modela tempo de espera não negativo quando os eventos seguem um processo de Poisson homogêneo com taxa λ por unidade de tempo.

- Densidade: $f(x)=\lambda e^{-\lambda x}$, para $x\geq 0$.
- $P(X\leq x)=1-e^{-\lambda x}$; $P(X>x)=e^{-\lambda x}$.
- Média: $1/\lambda$; variância: $1/\lambda^2$; desvio padrão: $1/\lambda$.

Em `scipy.stats.expon`, `scale` é **1/λ**, não λ:

```python
from scipy.stats import expon

expon.cdf(x, scale=1/lambda_)
expon.sf(x, scale=1/lambda_)
```

Funções de distribuição no SciPy

Para distribuições contínuas, os métodos mais usados são:

- `pdf(x, ...)` : densidade no ponto (não é a probabilidade de um ponto específico).
- `cdf(x, ...)` : área acumulada à esquerda, $P(X \leq x)$.
- `sf(x, ...)` : área à direita, $P(X > x)$; frequentemente mais precisa que `1-cdf` .
- `ppf(q, ...)` : quantil cuja área acumulada é q .
- `isf(q, ...)` : quantil cuja área à direita é q .

4. Amostragem e Teorema Central do Limite

- **População:** conjunto completo que se deseja estudar.
- **Amostra:** parte observada da população.
- **Parâmetro:** característica da população (μ , σ , p).
- **Estatística:** medida calculada da amostra ($\bar{x}$, s , $\hat{p}$).
- **Distribuição amostral:** distribuição de uma estatística ao repetir amostragens do mesmo tamanho.
- **Erro padrão da média:** $\sigma_{\bar{x}} = \sigma/\sqrt{n}$ (ou $s/\sqrt{n}$ quando σ é desconhecido).

O TCL diz que, com variância finita, a distribuição das médias amostrais tende à normal à medida que n cresce. Nas aulas usa-se frequentemente $n \geq 30$ como referência para amostras grandes; não é uma garantia universal, especialmente para dados muito assimétricos ou com dependência. Amostras maiores reduzem o erro padrão.

5. Testes de hipótese: roteiro e interpretação

Roteiro

1. Identifique o parâmetro e a pergunta: diferença, aumento, redução ou igualdade?
2. Escreva H_0 (hipótese nula, normalmente igualdade) e H_a (o efeito/diferença que se quer investigar).
3. Defina α antes do teste. Confiança de 95% corresponde a $\alpha = 0{,}05$.
4. Verifique os pressupostos e selecione o teste.
5. Calcule a estatística e o p-valor (ou compare com valor crítico).

6. Conclua no contexto da pergunta, informando evidência, direção e nível α.

Caudas e decisões

- H_a: parâmetro $\neq$ valor : **bicaudal**.
- H_a: parâmetro $>$ valor : **unicaudal à direita**.
- H_a: parâmetro $<$ valor : **unicaudal à esquerda**.
- Regra do p-valor: se $p \leq \alpha$, rejeite H_0; se $p > \alpha$, **não rejeite** H_0.
- Não rejeitar H_0 não prova que ela seja verdadeira nem que os grupos sejam iguais; significa que a amostra não forneceu evidência suficiente contra ela.
- O p-valor não é a probabilidade de H_0 ser verdadeira. É a probabilidade, supondo H_0, de obter resultado tão extremo quanto ou mais extremo que o observado.

Valores críticos Z comuns:

Situação	Valor crítico
$\alpha=0,05$, bicaudal	±1,96
$\alpha=0,01$, bicaudal	±2,576
$\alpha=0,10$, bicaudal	±1,645
$\alpha=0,05$, unicaudal	1,645 (direita) ou −1,645 (esquerda)

Erros: tipo I (α) rejeitar H_0 quando ela é verdadeira; tipo II (β) não rejeitar H_0 quando ela é falsa. Potência é $1-\beta$. Significância estatística não é necessariamente importância prática: considere tamanho do efeito, escala e contexto.

6. Qual teste usar?

Pergunta / dados	Teste típico	Pontos de atenção
Uma média vs valor; σ populacional conhecido ou aproximação normal adequada	Z de uma amostra	Amostra aleatória/independente; $z=(\bar{x}-\mu_0)/(\sigma/\sqrt{n})$
Uma média vs valor; σ desconhecido	t de uma amostra	$t=(\bar{x}-\mu_0)/(s/\sqrt{n})$, $gl=n-1$; para n pequeno, normalidade é importante

Pergunta / dados	Teste típico	Pontos de atenção
Duas médias de grupos independentes	t independente (Welch como padrão seguro)	Amostras independentes; Welch não exige variâncias iguais
Duas médias independentes, variâncias iguais justificadas	t pooled	Variância combinada; $gl=n_1+n_2-2$
Duas médias, desvios populacionais conhecidos ou amostras grandes conforme a abordagem da aula	Z de duas amostras	$z=(\bar{x}_1-\bar{x}_2)/\sqrt{(\sigma_1^2/n_1+\sigma_2^2/n_2)}$
Mesmas pessoas/unidades medidas duas vezes ou pares correspondentes	t pareado	Calcule diferenças d_i ; teste a média das diferenças
Uma proporção vs valor p_0	Binomial exato ou Z aproximado	Resultado binário; para aproximação normal, verificar sucessos e fracassos esperados
Duas proporções independentes	Z de duas proporções	Sob $H_0:p_1=p_2$, use proporção combinada; verificar contagens esperadas
Uma variância vs valor	Qui-quadrado (χ^2)	Dados normais e amostra aleatória/independente
Duas variâncias	F	Dados normais; ordenar variâncias e respeitar graus de liberdade/caudas
Três ou mais médias independentes	ANOVA de um fator	Teste global; se significativo, faça comparação pós-teste (por exemplo, Tukey)

Não escolha o teste apenas pelo tamanho n : considere desenho (pareado/independente), tipo de variável, parâmetro, distribuição e pressupostos. Para proporções, a condição usual de aproximação é haver ao menos 10 sucessos e 10 fracassos esperados **em cada grupo**, calculados sob a hipótese nula quando apropriado; caso contrário, considere um teste exato.

7. Testes de hipótese

Uma média

Z de uma amostra: $z=(\bar{x}-\mu_0)/(\sigma/\sqrt{n})$.

t de uma amostra: $t=(\bar{x}-\mu_0)/(s/\sqrt{n})$, com $gl=n-1$.

```
from scipy import stats
from statsmodels.stats import weightstats

stats.ttest_1samp(amostra, popmean=valor_h0)
weightstats.ztest(amostra, value=valor_h0)  # Z de uma amostra
```

Duas médias independentes

Hipótese bilateral usual: $H_0: \mu_1=\mu_2$; $H_a: \mu_1\neq\mu_2$.

Para t de Welch, a forma direta é:

```
stats.ttest_ind(grupo1, grupo2, equal_var=False, nan_policy="omit")
```

`equal_var=False` não pressupõe variâncias iguais. Use `equal_var=True` apenas quando a variância combinada for adequada ao desenho e às premissas. Z de duas amostras usa o erro padrão $\sqrt{(\sigma_1^2/n_1 + \sigma_2^2/n_2)}$.

t pareado

Defina as diferenças de cada par, por exemplo `d = depois - antes` . Então:

- $t = \bar{d} / (s_d/\sqrt{n})$, $gl=n-1$.
- $H_0: \mu_d=0$; alternativa bilateral: $H_a: \mu_d\neq0$.

```
stats.ttest_rel(antes, depois, nan_policy="omit")
# ou: stats.ttest_1samp(depois - antes, popmean=0)
```

Uma proporção

Se `x` sucessos em `n` observações, $\hat{p}=x/n$. Para Z aproximado:

$z = (\hat{p}-p_0) / \sqrt{[p_0(1-p_0)/n]}$.

```
stats.binomtest(x, n, p=p0, alternative="two-sided")
# alternative: "two-sided", "less" ou "greater"
```

O binomial exato não depende da aproximação normal.

Duas proporções

Com x_1 sucessos em n_1 e x_2 em n_2, sob $H_0: p_1 = p_2$, a proporção combinada é $\bar{p} = (x_1+x_2)/(n_1+n_2)$. O erro padrão nulo é $\sqrt{[\bar{p}(1-\bar{p})(1/n_1+1/n_2)]}$.

```
from statsmodels.stats.proportion import proportions_ztest

sucessos = [x1, x2]
tamanhos = [n1, n2]
proportions_ztest(sucessos, tamanhos, alternative="two-sided")
```

Uma variância: qui-quadrado

Para testar $H_0: \sigma^2 = \sigma_0^2$, assumindo população normal:

- $\chi^2 = (n-1)s^2/\sigma_0^2$
- Graus de liberdade: $n-1$.

```
from scipy.stats import chi2

chi2_calculado = (n - 1) * s**2 / sigma0**2
chi2.isf(alpha, df=n - 1)  # valor crítico na cauda direita
```

O teste é assimétrico; para duas caudas, dividir α entre as duas e usar os quantis apropriados. Alternativamente, construa p-valor com `chi2.cdf` / `chi2.sf` conforme a cauda.

Duas variâncias: F

Para $H_0: \sigma_1^2 = \sigma_2^2$, uma estatística comum é $F = s_1^2/s_2^2$, com graus de liberdade $gl_1 = n_1 - 1$ e $gl_2 = n_2 - 1$. Se colocar a maior variância no numerador, $F \geq 1$, ajuste os graus de liberdade e o valor crítico à convenção da tabela/teste. Requer normalidade; é sensível a desvios dessa premissa.

ANOVA de um fator

Compara simultaneamente $k \geq 3$ médias:

- $H_0: \mu_1 = \mu_2 = ... = \mu_k$.
- H_a : pelo menos uma média difere.
- A estatística F compara variabilidade entre grupos com variabilidade dentro dos grupos.
- Se $p \leq \alpha$, a evidência indica que **ao menos uma** média difere; ANOVA, por si só, não identifica quais.
- Após ANOVA significativa, use comparações múltiplas (ex.: Tukey) para localizar diferenças.
- ANOVA clássica pressupõe observações independentes, resíduos aproximadamente normais e variâncias semelhantes.

```
stats.f_oneway(grupo1, grupo2, grupo3)
stats.tukey_hsd(grupo1, grupo2, grupo3)  # pós-teste quando adequado
```

8. Python para análise de dados

Preparação e estruturas

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

df = pd.read_csv("arquivo.csv")
df = pd.read_csv("arquivo.csv", sep=";")  # CSV separado por ponto e vírgula
```

- Listas: `itens=[1, 2, 3]` ; índice começa em zero (`itens[0]`); fatia `itens[1:3]` exclui o limite final; `append` , `pop` , `sort` .
- Dicionários: mapeiam chave para valor, por exemplo `nomes={1: "Primavera", 2: "Verão"}` .
- `range(inicio, fim, passo)` gera inteiros até, mas sem incluir, `fim` .
- NumPy: `np.array` , `np.arange` , `np.linspace` , `np.zeros((linhas, colunas))` , `np.ones(...)` , `reshape` .
- Indexação NumPy: `a[índice]` ; matriz `m[linha, coluna]` ; fatias `m[:, 1:3]` ; filtro booleano `a[a >= 60]` .

Inspecionar e limpar

```
df.shape                # (linhas, colunas)
df.head()               # primeiras linhas
df.dtypes               # tipos
df.info()               # tipos e valores não nulos
df.describe()           # resumo numérico
df.isna().sum()         # ausentes por coluna
df = df.dropna(subset=["coluna1", "coluna2"]).copy()
```

Selecionar, filtrar, ordenar e criar colunas

```
df["coluna"]                        # Series
df[["coluna1", "coluna2"]]          # DataFrame
df.loc[rotulo, ["coluna1"]]         # por rótulo
df.iloc[0:5, 0:2]                   # por posição

filtro = (df["cidade"] == "Albany") & (df["quartos"] == 3)
recorte = df[filtro]                 # use parênteses em cada condição
df[df["categoria"].isin(["A", "B"])]
df[df["valor"].between(10, 20)]
df = df.sort_values("valor", ascending=False)
df.nlargest(10, "valor")
df["lucro"] = df["receita"] - df["custo"]
```

Para combinar condições: `&` significa E; `|` significa OU; `~` nega uma condição. Em pandas, não use `and` / `or` com Series.

Agrupar e resumir

```
df.groupby("grupo")["valor"].mean()

resumo = df.groupby("grupo").agg(
    quantidade=("valor", "size"),
    media=("valor", "mean"),
    desvio=("valor", "std")
)

tabela = pd.crosstab(df["linha"], df["coluna"], margins=True)
proporcoes_linha = pd.crosstab(
    df["linha"], df["coluna"], normalize="index"
)
```

`normalize=True` normaliza pelo total geral; `"index"` normaliza cada linha; `"columns"` cada coluna. Para agregar um valor numérico em uma tabulação cruzada:

```
pd.crosstab(df["grupo"], df["categoria"],
            values=df["medida"], aggfunc="mean")
```

Gráficos comuns

```
df["valor"].hist(bins=15)                                   # histograma
df.boxplot(column="valor", by="grupo")                      # boxplot
sns.histplot(data=df, x="valor", bins=15, kde=True)
sns.boxplot(data=df, x="grupo", y="valor")
sns.countplot(data=df, x="categoria", hue="grupo")

resumo.plot(kind="bar", x="grupo", y="media")
resumo.plot(kind="line", marker="o")
tabela.plot(kind="bar", stacked=True)
tabela.plot(kind="area", stacked=True)

plt.title("Título")
plt.xlabel("Eixo X")
plt.ylabel("Eixo Y")
plt.tight_layout()
plt.show()
```

Pizza: use para poucas categorias que compõem um todo (`plt.pie(valores, labels=rotulos, autopct="%1.1f%%")`). Para comparação entre grupos, barras ou boxplots frequentemente deixam diferenças mais claras. Use `plt.subplot(linhas, colunas, posição)` para painéis.

Padrão de conclusão estatística

> Com `α=...` , o teste ... resultou em `p=...` . Como `p [≤ ou >] α` , [rejeitamos / não rejeitamos] H_0 . Há [evidência / evidência insuficiente] de que ... no contexto do problema. Isso não demonstra que H_0 seja verdadeira quando não rejeitada.

9. Exemplos de fluxo de trabalho

Filtrar, resumir e plotar

```
df = pd.read_csv("dados.csv")
recorte = df[(df["categoria"] == "A") & (df["valor"].notna())]
print(recorte["valor"].mean(), recorte["valor"].median(),
      recorte["valor"].std())

resumo = df.groupby("grupo").agg(
    quantidade=("valor", "size"),
    media=("valor", "mean")
)
resumo["media"].plot(kind="bar")
plt.tight_layout()
plt.show()
```

Comparar duas médias em dados de atividade

```
dados = df.dropna(subset=["salario", "usa_ferramenta"]).copy()
g1 = dados.loc[dados["usa_ferramenta"], "salario"]
g2 = dados.loc[~dados["usa_ferramenta"], "salario"]
resultado = stats.ttest_ind(g1, g2, equal_var=False)
print(resultado.statistic, resultado.pvalue)
```

Antes de concluir, verifique que os grupos são independentes, que a variável é numérica e que os valores ausentes foram tratados. A forma de indexar booleanos pode exigir conversão explícita se a coluna não for booleana.

10. Mapa do material original

O conteúdo cobre as aulas 00–23. Use os diretórios/notebooks originais para ver enunciados completos, células de código e exemplos visuais.

Aulas / material	Tópicos principais	Arquivos-base
00–01	Apresentação da disciplina; leitura de dados e inspeção inicial de DataFrame	Aula 00 - 01
02	Listas, dicionários e estruturas Python	Aula 02 - Lista
03	Média, moda, mediana, dispersão e quantis	Aula 03 - Media
04	Arrays NumPy, matrizes, indexação e operações	Aula 04 - Arrays
05	Probabilidade, simulações e regras de eventos	Aula 05 - Probabilidade
06–07	Distribuições binomial e Poisson	Aula 06 - Distribuicao Binomial; Aula 07 - Poisson
08	Series, DataFrames, leitura de CSV, seleção e agregação	Aula 08 - Series
09–11	Gráficos; linhas; tabelas cruzadas e	Aula 09 - Plotagem de Graficos; Aula 10 - Grafico de Linha; Aula 11 - CrossTab

Aulas / material	Tópicos principais	Arquivos-base
	normalização	
12–14	Normal, exponencial, distribuição amostral e TCL	Aula 12 - Distribuicao Normal; Aula 13 - Distribuicao Exponencial; Aula 14 - Teorema Central do Limite
15–23	Testes de hipótese, médias, proporções, variâncias e ANOVA	Aula 15 - Teste de Hipótese; Aula 16 - Testes T e Z de Uma Amostra; Aula 17 - Teste de Uma Proporção; Aula 18 - Teste de Variância; Aula 19 - Testes t e Z de Duas Amostras; Aula 20 - Teste t Pareado; Aula 21 - Teste de Duas Proporções; Aula 22 - Teste de Variâncias; Aula 23 - ANOVA
Atividades	Descritiva, visualização, filtragem, resumo, proporções e testes	Atividade Avaliativa; Atividade em Sala 01; Atividade em Sala 02

Bases CSV incluídas

- **Atividade Avaliativa:** `nba2k.csv`, `day.csv`, `student-por.csv`.
- **Atividade em Sala 01:** `movies.csv`, `real_state_ny.csv`.
- **Atividade em Sala 02:** `Ames.csv`, `stackoverflow_2024.csv`.
- **Aula 08:** `Piece_Dim.csv`.
- **Aulas 09–11:** `bloodPressure.csv`, `ebola.csv`, `GDP_by_Country.csv`, `survey2021.csv`, `washingtonDCCrime.csv`, `apple.csv`, `CPI_2014.csv`, `lifeExpectancy.csv`, `migrants.csv`, `tesla.csv`, `nobel_prize.csv`.
- **Aulas 16–23:** `Machine1.csv`, `Machine2.csv`, `Smokers.csv`, `Two_Machines.csv`, `bike_sharing_hour.csv`.
- **Imagens de atividade:** `Q2.png`, `Q4.png`, `Q5.png`, `Q6.png`, `Q7.png` em Atividade em Sala 01.

11. Observações para evitar erros

1. Confirme se a pergunta pede **exatamente**, **no máximo**, **pelo menos**, **menor que** ou **maior que**: isso determina `pmf`, `cdf`, `sf` e a inclusão do limite.

2. Verifique unidades e escala de λ (eventos por minuto, hora etc.) e de σ (mesma unidade da variável).
3. Em hipóteses, escreva primeiro a alternativa em palavras para escolher corretamente a cauda.
4. A confiança 95% equivale a $\alpha=0,05$; não significa que há 95% de chance de H_0 ser verdadeira.
5. Para testes t, `equal_var=False` em SciPy executa Welch; `ttest_rel` é para pares, e `ttest_ind` para grupos independentes.
6. Teste significativo não informa sozinho o tamanho nem a relevância prática da diferença. ANOVA significativa não identifica, sem pós-teste, quais grupos diferem.
7. Em gráficos, informe título, eixos, unidades e categorias. Em tabelas cruzadas, confira se a normalização foi pelo total, linha ou coluna.
8. Em filtros pandas, use parênteses nas comparações e `&` / `|` ; trate NaN antes de testes e cálculos que dependam de amostras completas.